Agentic AI 1: LLM foundations and the agent loop — Basic

Portfolio projects · lessons 01–25

Each takes 1–3 days and ends with something to show an employer.

Project 1: Scout CLI: a streaming, cost-aware research chatbot with tools

Goal: Turn the running example into a polished command-line assistant over your own notes that streams answers, calls search and calculator tools (in parallel when it can) and prints exact cost per turn, showing you understand what every API field means.

Uses lessons: 06, 07, 08, 11, 12, 13, 17, 18, 19, 22, 25

1. Build a corpus of 40 short markdown notes (100-300 words each) on one subject you know, and a search_notes(query, k=3) tool using keyword scoring over them.

2. Add a calculator tool built on a safe AST evaluator (no eval) that supports + - * / ** and parentheses, with 15 unit tests including malicious inputs like __import__.

3. Write the chat loop with the google-genai SDK: system instruction, full history resent each turn, streamed output, and model id plus prices pinned in one config.py.

4. Handle one tool round per user turn: run every function call the model returns concurrently with asyncio.gather, send all results back in one message, then stream the final answer.

5. After each turn print input, output and total tokens, turn cost and running session cost from usage, and the stop reason; exit cleanly on max_tokens, safety blocks and empty replies.

6. Record a 20-question scripted session with llm_replay, then measure time-to-first-token (median and p95), cost per question and the share of questions answered with a correct note citation.

7. Add a --no-history flag and a 10-turn demo that shows the bot forgetting a fact from turn 2, with the token-per-turn curve that explains why resending history grows cost roughly quadratically.

Deliverable: Public repo with the CLI, notes corpus, recorded cassettes so `python scout.py --replay` runs with no key, pytest suite, and a README with a demo GIF and a table of measured TTFT, cost per question, citation accuracy and parallel-vs-sequential tool latency.

Stretch: Run the same CLI against a local Ollama model through the OpenAI-compatible endpoint and report where its tool calls break. · Add a token budget that trims the oldest turns before each call and show the cost curve flattening from quadratic to linear.

Project 2: Resilient LLM client with a provider failure lab

Goal: Ship a small, tested wrapper library that survives 429s, 529s, timeouts and truncated streams correctly, and prove it with a fake provider and a measured before/after failure table, the kind of reliability work platform teams value.

Uses lessons: 11, 12, 13, 14, 15, 17

1. Write a fake HTTP provider (FastAPI or http.server) with modes for 429 with retry-after, 429 without it, spend-cap 429, 529 overloaded, 500, read timeout, mid-stream disconnect and a tool call cut off mid-JSON at max_tokens.

2. Implement call_with_retry with jittered exponential backoff (base 0.5 s, cap 20 s, max 5 attempts) that honours retry-after and never retries 400s or spend-cap errors.

3. Classify every outcome into a typed result (ok, retryable_exhausted, fatal, truncated, blocked) and log request id, attempt count, latency and stop reason as one JSON line per call.

4. Write 20 pytest cases, one or more per failure mode, using a seeded random generator so backoff timings are deterministic.

5. Run a 1,000-request load against the fake provider with 30% injected failures, comparing no retries, naive fixed 1 s retries and your client on success rate, p50/p95 latency and total retry count.

6. Point the same client at real Gemini under llm_replay for 10 normal calls to show it adds no overhead on the happy path.

Deliverable: Repo with the client package, the fake provider, a `make drill` target that prints the before/during/after error table, and a README reporting measured success rate and latency for the three retry strategies.

Stretch: Add a client-side token-bucket rate limiter and show it removes most 429s at a 2x overload. · Resume a stream after a mid-stream disconnect by replaying the request and de-duplicating already-printed text.

Project 3: Support-ticket triage: structured extraction with measured reliability

Goal: Apply structured output to a realistic customer-support workload, extracting typed fields from messy tickets and proving with numbers which extraction method is reliable and cheap enough to ship.

Uses lessons: 12, 17, 19, 20, 21, 24

1. Assemble 200 support tickets (public datasets such as the Bitext customer-support set, or written by you) and hand-label 60 with category (8 classes), priority, product, refund_requested and a one-line summary.

2. Define the schema as a Pydantic model with enums and length limits, and export its JSON Schema.

3. Implement four extractors: prompt-only 'reply in JSON', JSON mode, schema-constrained decoding via response_schema, and a forced tool call with tool_choice set to the extract tool.

4. Wrap each extractor in Pydantic validation with up to 2 retries that feed the validation error back to the model, and count how often each retry fires.

5. Improve the prompt with a role, XML-tagged ticket text and 4 labelled examples, and measure the before/after change on the 60 labelled tickets.

6. Report per method: valid-JSON rate, schema-valid rate after retries, category accuracy and macro-F1, cost per 1,000 tickets and median latency, on two model sizes.

Deliverable: Repo with the labelled dataset, extractors, an evaluation script that runs from cassettes with no key, and a README with the comparison table, a confusion matrix and a one-paragraph recommendation for which method and model to ship.

Stretch: Add a 'needs_human' flag driven by a confidence field and measure precision of escalations. · Re-run the evaluation on a local Ollama model and report the quality and cost gap.

Project 4: Model bake-off: local vs hosted tool calling for a small-business assistant

Goal: Build a reproducible benchmark that picks a model for a booking assistant (check availability, quote a price, book a slot) on tool-call accuracy, latency and cost, the evidence-based model selection teams actually need.

Uses lessons: 03, 04, 10, 11, 12, 16, 17, 22, 23, 24, 25

1. Define three tools for a hair salon: check_availability(date, service), quote_price(service, stylist_level) and book_slot(date, time, service, name), with strict JSON schemas.

2. Write 50 test prompts with expected tool name and arguments, including 10 that need two parallel calls and 10 that need no tool at all.

3. Run each prompt against at least three models: a small Ollama model (e.g. a 3-8B instruct model), a Gemini flash-lite tier and a larger Gemini flash tier, with model ids pinned in one config.

4. Score tool-selection accuracy, exact-argument match, schema-valid arguments, false tool calls on the no-tool prompts and parallel-call completeness.

5. Measure time-to-first-token and total latency with streaming, plus input and output tokens counted before and after each call, and convert to cost per 1,000 requests from a dated price table.

6. Write the same single tool round trip once in Anthropic, OpenAI and Gemini wire formats and record which mistakes (missing or misplaced results) produce which 400s.

7. Run temperature 0 five times per prompt on the local model and report how often outputs differ.

Deliverable: Repo with the benchmark harness, test set and cassettes, and a README with a results table (accuracy, latency p50/p95, cost per 1,000), a cost-vs-accuracy chart and a written recommendation of which model to ship and why.

Stretch: Add a fallback router that tries the local model first and escalates to the hosted model when arguments fail validation, and measure the blended cost and accuracy. · Count tool-definition token overhead for each model and show how it changes with 3 versus 12 tools.